

Introduction to Machine Learning

by Khallil Ebrahim Benyattou

すこ
少しずつ

January 7, 2026

This page intentionally left almost blank.

Contents

Chapter 1: Introduction	
1.1 Paradigms	5
1.2 Tasks and Models	5
Chapter 2: Supervised Learning	
2.1 Probability to Model Uncertainty	9
Probabilistic Model.	
2.2 Loss and Risk	10
Chapter 3: Regression	
3.1 Setup	12
3.2 Estimating the Regression Function	12
3.3 Linear Regression	14
Extra Assumptions; Ordinary Least Squares (OLS); Gradient Descent; Stochastic Gradient Descent.	
3.4 Locally Weighted Linear Regression	21
3.5 Normality Assumptions (Ordinary Linear Regression)	22
Consequences of joint conditional normality of our errors; Maximum Likelihood Estimation.	
Chapter A: Matrix Calculus	

Introduction

[The] human perception system is rife with all ways of getting it wrong. [...] All it takes is a few sketches that are cleverly done and your brain can't figure it out! So, we are poor data-taking devices. That's why we have such a thing as science. Because we have machines that don't care what side of the bed they woke up (on) in the morning. [...] They'll get the data right.

Neil deGrasse Tyson, 2009
@ **Cosmic Quandaries**

An **algorithm** is an unambiguous sequence of instructions designed to complete a task. For simple tasks, it's relatively easy to manufacture an appropriate (and perhaps even optimal) algorithm. For more complicated tasks, it's often more practical to help the computer "learn" how to develop its own algorithm. I define **learning** as the process of turning information into knowledge and understanding.

Machine learning (ML) is the name given to the study of computer algorithms that improve *automatically* through experience (data).

An early definition of ML can be attributed to Arthur Samuel (1901–1990) — a pioneer of artificial intelligence research:

[Machine Learning is] a field of study that gives computers the ability to learn without being explicitly programmed.

Arthur Samuel, 1959

One of the first examples of a computer learning a task without being explicitly programmed was Samuel's own work on simulating the game of checkers. The computer even ended up surpassing Samuel himself. Nowadays, it isn't so surprising that computers are able to surpass human-level performance (on narrow tasks) but it was a real doozie back in the day.

A formal definition of the types of algorithms studied in machine learning is attributed to computer scientist Tom Mitchell:

A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks T , as measured by P , improves with experience E .

Tom M. Mitchell, 1997

e.g. For the task T of playing checkers, one could define the performance measure P as the percentage of games won against opponents, and the training experience E could be measured by the number of games simulated by the computer against itself. More concretely:

Experience E presents itself in the form of data. Data are observations. Thus, machine learning is a subfield of computer science that focuses on the study of algorithms that can learn from data and make predictions.

So far we've been very general and it helps to establish some form of taxonomy in the field of machine learning.

1.1 Paradigms

At a high level, a **paradigm** in machine learning is the broader philosophy or theoretical framework under which an algorithm is trained to solve a particular task. The paradigm one works in is informed by the type (structure and availability) of data (also called **training data**) presented and the nature of the feedback the algorithm receives i.e. how, if, or when the algorithm is informed about its performance. The broadest paradigms¹ are:

- **Supervised learning**
 - Type of data: A static (fixed) collection of labelled² data.
 - Feedback: Explicit and correct labels to compare our algorithm's predictions against.
- **Unsupervised learning**
 - Type of data: Static, unlabelled data.
 - Feedback: No explicit feedback and the algorithm must come to its own conclusions about the pattern or structure of the input data.
- Semi-supervised learning
 - Type of data: Static — a combination of labelled and unlabelled data.
 - Feedback: Explicit labels for a portion of the data (as in supervised learning) but the algorithm is designed to (often) infer patterns about the unlabelled data from the labelled data.
- **Reinforcement learning** is based on an agent-environment interaction framework driven by reward maximisation (or penalty minimisation).
 - Type of data: Dynamic — a set of observations (states, actions and transition probabilities) and rewards (and discounts for future reward value) that evolve over time.
 - Feedback: Rewards or penalties over time (immediate or delayed) based on the agent's sequential actions.

1.2 Tasks and Models

Suppose like before, that we are given a set of data and a task to accomplish within a particular paradigm of machine learning. The task is to investigate some real-life/natural phenomenon e.g. a *binary* classification task on e-mail data organises e-mails into *two* distinct groups — spam and not spam.

Oftentimes, such problems are particularly complex. To this end, it's beneficial to make some assumptions (influenced by either knowledge a priori or through exploratory data analysis) about which features/components are relevant, and how they relate to each other:

¹There are more emerging paradigms but these are the broadest ones.

²The data presents itself as a set of (feature vector, label) pairs.

In general parlance, a **model** is a mathematical, logical or physical representation of a process, concept or item. We use models to idealise complex phenomena and extract important information. The value of a model is that it approximates reality in a way that's useful.

All models are wrong, but some are useful.

George Box (probably)

Simpler and less flexible models tend to be easier to interpret. The tradeoff of a simple model is that it's less likely to capture enough of a phenomenon's inherent complexity — the resulting errors in calculation are called **bias**. There's an important balance to strike between flexibility and interpretability.

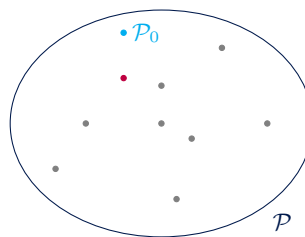
The term **machine learning model** is perhaps the most ambiguous term available to man. I'll try to be specific:

- A **probabilistic model** is the name given to the assumptions made about a process that generates training data.
- A **training/learning algorithm** is the procedure that uses the training data to learn a suitable function.
- I reserve the term **(machine learning) model** to refer to the **learned function** that has been **output** by a training algorithm.

Mathematics and statistics are natural languages to formalise a model.

A *mathematical model* is a description of a system using mathematical concepts and language. A *statistical model* is a type of mathematical model which embodies a set of statistical assumptions³ concerning the generation of sample data. Statistical models differ from other mathematical models by being non-deterministic. Thus, in a statistical model, some of the variables don't have specific values and instead have probability distributions — these variables are *stochastic*.

Formally, a statistical model can be thought of as a pair $(\Omega, \mathcal{P})$ where Ω is a set of possible observations (the outcome/sample space) and $\mathcal{P}$ is a set of probability distributions on Ω . The thinking behind this definition is that there exists a true probability distribution $\mathcal{P}_0$ that dictates the generation of the sample data that's been collected. $\mathcal{P}$ is typically (always?) parameterised in some way so we can write $\mathcal{P} = \{\mathcal{P}_\theta : \theta \in \Theta\}$. This collection contains a **distribution** that adequately approximates $\mathcal{P}_0$.



A statistical model $(\Omega, \mathcal{P} = \{\mathcal{P}_\theta : \theta \in \Theta\})$ is called *parametric* if Θ has finite dimension i.e. $\Theta \subseteq \mathbb{R}^k$ where $k \in \mathbb{Z}^+$. k is called the dimension of the model. A model is called *non-parametric* if Θ is infinite-dimensional.

³Usually specified as a mathematical relationship between random variables and non-random variables.

Most of the recent economic value generated by machine learning is through supervised learning but there are important use-cases for unsupervised learning.

Andrew Ng (2018)

We begin with supervised learning ↓

CHAPTER 2

Supervised Learning

Supervised learning is the paradigm of machine learning under which we use labelled training data to employ an algorithm that outputs a machine learning model which can correctly classify an unseen input, or predict an outcome.

The labelled training data is a set of $n \in \mathbb{Z}$ (object, label) pairs

$$\mathcal{T} := \{(x^{(i)}, y^{(i)})\}_{i=1}^n$$

We call this set the **training set**. Each pair $(x^{(i)}, y^{(i)})$ is called a **training example**:

- Each $x^{(i)}$ is an **input vector** of dimension p which contains information about each object:

$$x^{(i)} = \begin{bmatrix} x_1^{(i)} \\ \vdots \\ x_p^{(i)} \end{bmatrix}.$$

- In many applications, there is typically some form of feature-extraction that is performed on the original data variables. This results in a **feature vector**:

$$\phi(x^{(i)}) = \begin{bmatrix} \phi_1(x^{(i)}) \\ \vdots \\ \phi_M(x^{(i)}) \end{bmatrix},$$

where each feature $\phi_j(x^{(i)})$ is the result of evaluating the j^{th} basis function ϕ_j at $x^{(i)}$. Each ϕ_j can be non-linear so this allows for greater flexibility in modelling.

- $y^{(i)}$ is called the **label** or **response** corresponding to $x^{(i)}$.

Let's say the input vectors $x^{(i)}$ live in a space $\mathcal{X}$ and the labels $y^{(i)}$ live in $\mathcal{Y}$. Then, $\mathcal{T} \subseteq \mathcal{X} \times \mathcal{Y}$.

The goal of a supervised learning algorithm is to use a set of training data $\mathcal{T}$ to learn a function $h: \mathcal{X} \rightarrow \mathcal{Y}$ that predicts the value of $y \in \mathcal{Y}$ corresponding to a given input feature vector $x \in \mathcal{X}$. Such a function h has historically been called a **hypothesis**.

A successful (and if we're lucky, optimal) h will exhibit the following two properties:

- For each example $(x^{(i)}, y^{(i)})$, the prediction $h(x^{(i)})$ is a “good” estimate for the true label $y^{(i)}$.
- The estimate h also generalises well in the sense that its output $h(x)$ for some unseen vector $x \in \mathcal{X}$ is a “good” predictor for its corresponding expected value of $y \in \mathcal{Y}$ given x .

A supervised learning problem is called

- a **regression problem** if $\mathcal{Y} = \mathbb{R}$, or
- a **classification problem** if the output label y can only take on a discrete number of values i.e. $\mathcal{Y} = \{c_1, \dots, c_K\}$ for $K < \infty$. In this case, h is typically called a **classifier**.

2.1 Probability to Model Uncertainty

When it comes to deciding on how to represent h , it's always a good start to plot the data $\mathcal{T}$ and see if there's an obvious relationship between the inputs $x_j^{(i)}$ and the response $y^{(i)}$ e.g.

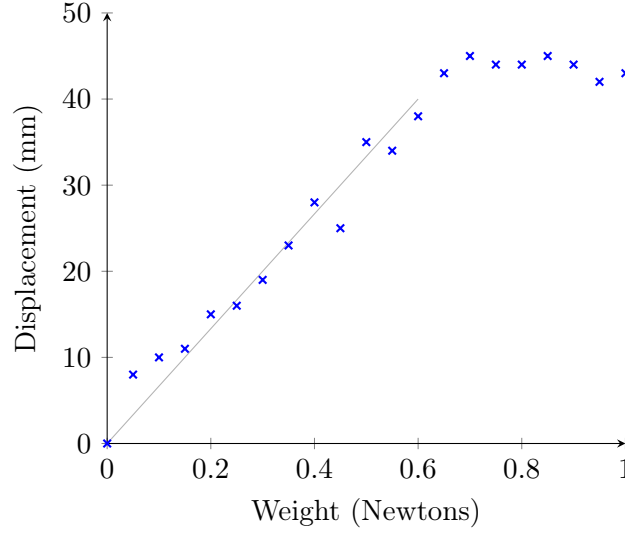


Figure 2.1: Plotting the relationship between the displacement of a spring and the weight placed on it to investigate Hooke's Law. In this case, weight is $\mathcal{X}$ and displacement is $\mathcal{Y}$.

For the above data, we could conceivably use some form of positive square function for h . There are many such functions, and we could naturally parameterise this family of functions as a collection $\mathcal{H} := \{h_{\theta} : \theta \in \Theta\}$ depending on said parameter $\theta \in \Theta$.

Such a $\mathcal{H}$ is called a **hypothesis class** — it's clear that the choice of $\mathcal{H}$ has far-reaching consequences on all subsequent analysis.

The data collection method is almost never perfect so there will be some random error in the observed values. This can be seen in the above figure by the **data** being distributed somewhat randomly about a positive linear trend (represented by the **straight line**) which plateaus off around 0.6 Newtons. Modelling such uncertainty can be done by introducing the language of statistics/probability.

2.1.1 PROBABILISTIC MODEL

We require a few definitions to properly state the assumptions being made.

Definition 2.1.1

- Let $(\Omega, \mathcal{F}, \mathbb{P})$ be a probability space. This will model the randomness governing the phenomenon from which we wish to sample our data.
- Let $(E, \mathcal{E})$ be another measurable space. A **random element** is an $(\mathcal{F}, \mathcal{E})$ -measurable map $X: \Omega \rightarrow E$ i.e. $\forall B \in \mathcal{E}, X^{-1}(B) \in \mathcal{F}$. Such maps formalise the process of drawing a single observable (number, vector etc. e.g. height) corresponding to an outcome (e.g. the drawing of a student from a class).
 - If $(E, \mathcal{E}) = (\mathbb{R}, \mathcal{B}_{\mathbb{R}})$, then X is a real-valued **random variable**.
 - If $(E, \mathcal{E}) = (\mathbb{R}^n, \mathcal{B}_{\mathbb{R}^n})$, then X is a real-valued **random vector**.
- A **random sample** of size n is a collection of random variables $X_i: (\Omega, \mathcal{F}) \rightarrow (E, \mathcal{E})$ defined on the same probability space $(\Omega, \mathcal{F}, \tilde{\mathbb{P}})$ that are mutually $\tilde{\mathbb{P}}$ -independent, and identically distributed with distribution $\mathbb{P}$ s.t. $\tilde{\mathbb{P}} = \otimes_n \mathbb{P}$. We denote this by

$$X_i \stackrel{\text{i.i.d.}}{\sim} \mathbb{P}.$$

If we formalise the act of sampling one input-output pair (x, y) from said phenomenon by a random vector $(\mathbf{X}, Y): \Omega \rightarrow \mathcal{X} \times \mathcal{Y}$, then $\mathbb{P}_{\mathcal{X} \times \mathcal{Y}}$ is the name we give to the pushforward measure $\mathbb{P}_{(\mathbf{X}, Y)}$ of $\mathbb{P}$ via $(\mathbf{X}, Y)$ i.e. $\mathbb{P}_{\mathcal{X} \times \mathcal{Y}}$ is the assumed true joint distribution of the phenomenon that we sample from to generate our data $\mathcal{T}$. We make a simplifying assumption on the way we sample our training set:

- We can view our training set $\mathcal{T}$ as a realisation of a random sample

$$(\mathbf{X}^{(1)}, Y^{(1)}), \dots, (\mathbf{X}^{(n)}, Y^{(n)}) \stackrel{\text{i.i.d.}}{\sim} \mathbb{P}_{\mathcal{X} \times \mathcal{Y}}.$$

- Each $(\mathbf{X}^{(i)}, Y^{(i)}): \Omega \rightarrow \mathcal{X} \times \mathcal{Y}$ is a random variable and its realisation in our training set is $(x^{(i)}, y^{(i)})$.

2.2 Loss and Risk

Remarks 2.2.1 (Notational) Let $(\Omega, \mathcal{F}, \mathbb{P})$ be a probability space, and $X: \Omega \rightarrow \mathbb{R}$ be a random variable i.e. X is $\mathcal{F}$ -measurable.

- I use ordinary parentheses for the expectation of X :

$$\mathbb{E}(X) = \int_{\Omega} X(\omega) \, d\mathbb{P}(\omega),$$

- and square parentheses for random variables e.g.
 - The conditional expectation $\mathbb{E}[X | \mathcal{G}]$ of X given a sub- σ -algebra $\mathcal{G} \subseteq \mathcal{F}$. Any random variable Z is called a conditional expectation of X given $\mathcal{G}$ if Z is $\mathcal{G}$ -measurable, and satisfies the averaging property i.e. for every $B \in \mathcal{G}$:

$$\mathbb{E}(\mathbb{1}_B Z) = \mathbb{E}(\mathbb{1}_B \mathbb{E}(X | \mathcal{G})).$$

- The conditional probability of $A \in \mathcal{F}$ given $\mathcal{G}$ is denoted by $\mathbb{P}[A | \mathcal{G}]$
- I write $\text{Var}[X | \mathcal{G}]$ to represent the conditional variance (as seen later).

By virtue of the randomness in our assumptions so far, making (almost) any choice of hypothesis will yield predictions $h(x^{(i)})$ that fall short from the observed label $y^{(i)}$ i.e. we incur a loss. The **loss** of a particular hypothesis $h \in \mathcal{H}$ associated with a particular training example $(x^{(i)}, y^{(i)})$ is made precise as a non-negative real-valued function $L: \mathcal{H} \times \mathcal{T} \rightarrow [0, +\infty)$ defined by

$$(h, (x^{(i)}, y^{(i)})) \mapsto L(h(x^{(i)}), y^{(i)}).$$

In statistics, it's common practice to **average the losses** over an entire population — the deterministic number $R(h)$, called the **statistical risk**, is defined as the **expected** value of the chosen loss function of the hypothesis h with respect to the true distribution $\mathbb{P}_{\mathcal{X} \times \mathcal{Y}}$ of the (population) phenomenon under investigation:

$$\begin{aligned} R(h) &:= \mathbb{E}(L(h(\mathbf{X}), Y)) := \int_{\Omega} L(h(\mathbf{X}(\omega)), Y(\omega)) \, d\mathbb{P}(\omega) \\ &= \int_{\mathcal{X} \times \mathcal{Y}} L(h(\mathbf{x}), y) \, d\mathbb{P}_{\mathcal{X} \times \mathcal{Y}}(\mathbf{x}, y). \end{aligned}$$

A function that minimises the statistical risk would be the best hypothesis h available with respect to the given loss L . We denote this minimiser by

$$h^* := \underset{h}{\operatorname{argmin}} \mathbb{E}(L(h(\mathbf{X}), Y)).$$

In practice, we don't know the joint distribution $\mathbb{P}_{\mathcal{X} \times \mathcal{Y}}$ of our population so we can't compute $R(h)$ directly. Instead, we can use our aforementioned random sample to define an estimator called the **empirical risk**:

$$\hat{R}(h) = \frac{1}{n} \sum_{i=1}^n L(h(\mathbf{X}^{(i)}), Y^{(i)}).$$

The $(\mathbf{X}^{(i)}, Y^{(i)})$ are still i.i.d. with distribution $\mathbb{P}_{\mathcal{X} \times \mathcal{Y}}$ but now we have the possibility of using the realisations of the random sample (i.e. the training data $\mathcal{T}$) to compute the empirical risk

$$\hat{R}(h) = \frac{1}{n} \sum_{i=1}^n L(h(x^{(i)}), y^{(i)})$$

which is just the average loss on the training set.

CHAPTER 3

Regression

Let $\mathcal{X} = \mathbb{R}^p$ and $\mathcal{Y} = \mathbb{R} = (-\infty, +\infty)$.

Regression is a term used as a shorthand for regression analysis — a series of statistical techniques developed to investigate the relationship between a collection of (one or more) independent/predictor variables (the regressors) $X_1, \dots, X_p$, and a¹ real-valued dependent/response variable (the regressand) Y .

3.1 Setup

As established in the previous section, we assume that there is some true joint distribution between predictors and response, represented by $(\mathbf{X}, Y) \sim \mathbb{P}_{\mathcal{X} \times \mathcal{Y}}$. Since our goal is to optimally predict the response Y given the predictors $\mathbf{X}$, we let $f: \mathcal{X} \rightarrow \mathcal{Y}$ denote a deterministic function s.t.

$$f(\mathbf{X}) = \mathbb{E}[Y | \mathbf{X}].$$

Since $\mathbf{X}$ and $\mathbf{Y}$ are random variables², there will naturally be some error between the prediction $f(\mathbf{X})$ and Y . We denote this error by the random variable³

$$\varepsilon := Y - f(\mathbf{X}) = Y - \mathbb{E}[Y | \mathbf{X}].$$

It immediately follows that we may re-write this equality in the familiar form

$$Y = f(\mathbf{X}) + \varepsilon,$$

but more importantly it follows that

$$\begin{aligned} \mathbb{E}[\varepsilon | \mathbf{X}] &= \mathbb{E}[Y - f(\mathbf{X}) | \mathbf{X}] \\ &= \mathbb{E}[Y | \mathbf{X}] - \mathbb{E}[f(\mathbf{X}) | \mathbf{X}] \\ &= \mathbb{E}[Y | \mathbf{X}] - f(\mathbf{X}) \\ &= \mathbb{E}[Y | \mathbf{X}] - \mathbb{E}[Y | \mathbf{X}] \\ &= 0. \end{aligned}$$

This property $\mathbb{E}[\varepsilon | \mathbf{X}] = 0$ is called strict exogeneity. It means that our errors have zero mean conditional on the regressors $\mathbf{X}$.

3.2 Estimating the Regression Function

Since we don't know $\mathbb{P}_{\mathcal{X} \times \mathcal{Y}}$, we can't calculate $\mathbb{E}[Y | \mathbf{X}]$ directly. Instead, we must estimate f by choosing a hypothesis⁴ h from a (suitable) class of candidate functions $\mathcal{H}$ called the **hypothesis class**. Then we shall use our sampled data $\mathcal{T}$ to optimise h in a way that best approximates $\mathbb{E}(Y | \mathbf{X})$.

¹Strictly speaking, this would be a multivariable univariate regression — variable refers to predictors, and variate refers to responses.

²Does Y need to be L^1 for the conditional expectation to exist?

³Can this error be thought of as the randomness in Y , on average, that isn't captured by $\mathbf{X}$?

⁴We will use this function to make predictions for unseen examples $x \in \mathcal{X}$.

For regression, a common choice⁵ of loss function is the **squared/quadratic loss** $L(h(\mathbf{X}), Y) = (h(\mathbf{X}) - Y)^2$. For quadratic loss, the statistical risk of h is $R(h) := \mathbb{E}((Y - h(\mathbf{X}))^2)$ and is called the **mean squared error (MSE)** for short). The assumptions that Y and $h(\mathbf{X}) \in L^2$ are required for this minimisation problem to be meaningful.

A function that minimises the MSE would be the best hypothesis available under quadratic loss. This minimiser is denoted by

$$h^* := \operatorname{argmin}_h \mathbb{E}((Y - h(\mathbf{X}))^2).$$

Does such a function exist, and if so, which form does it take? By conditioning on $\mathbf{X}$, we can rewrite the MSE using the law of total expectation

$$\mathbb{E}((Y - h(\mathbf{X}))^2) = \mathbb{E}\left(\mathbb{E}((Y - h(\mathbf{X}))^2 \mid \mathbf{X} = x)\right).$$

It suffices to minimise the argument of the expectation on the RHS pointwise because given $\mathbf{X} = x$, $h(\mathbf{X})$ is equal to some constant $h(x) =: c$ so we concern ourselves with:

$$h^*(x) = \operatorname{argmin}_c \mathbb{E}[(Y - c)^2 \mid \mathbf{X} = x]$$

This argmin is actually equal to the conditional expectation $\mathbb{E}[Y \mid \mathbf{X} = x]$ of Y given $\mathbf{X} = x$.

There are three ways to verify this:

1. Add and subtract $\mathbb{E}(Y \mid \mathbf{X})$ from the statistical risk expression and expand.
2. Observe that

$$L^2(\Omega, \mathbb{P}) := \{\text{random variables } Y : \Omega \rightarrow \mathbb{R}$$

such that $\mathbb{E}(|Y|^2) < \infty\}$ is a vector space and when equipped with the inner product $\langle X, Y \rangle = \mathbb{E}(XY)$ becomes a Hilbert space. Then we can use the theory of orthogonal projections to prove that $\mathbb{E}(Y \mid \mathbf{X}) := \mathbb{E}(Y \mid \sigma(\mathbf{X}))$ (where $\sigma(X)$ is the sigma-algebra generated by $\mathbf{X}$) is equal to $\pi_{\sigma(\mathbf{X})}$ ($\mathbb{P}$ -almost surely) where

$$\pi_{\sigma(\mathbf{X})} : L^2(\Omega, \mathcal{A}, \mathbb{P}) \rightarrow L^2(\Omega, \sigma(\mathbf{X}), \mathbb{P})$$

is the projection map defined by

$$\|Y - \pi_{\sigma(\mathbf{X})}(Y)\| = \inf\{\|Y - Z\| : Z \in L^2(\Omega, \sigma(\mathbf{X}), \mathbb{P})\}$$

where $\sigma(\mathbf{X})$ is a sub-algebra of $\mathcal{A}$. In summary, this is a fancy way to say that the geometry of Hilbert space tells us that minimisation in said space becomes a problem of orthogonal projection.

3. A differentiation argument.

This minimiser $h^*(x) = \mathbb{E}[Y \mid \mathbf{X} = x]$ coincides with f , and so the terms **regression function** and **conditional expectation function (CEF)** are synonymous.

As stated at the top of this section, we don't know the (joint and hence) conditional distribution of Y given $\mathbf{X}$. We can get around this by modelling the regression function as some simple functional form h and then use the training data to minimise the empirical risk instead. In the case that the regression function h is parametric i.e. $h = h_{\theta}$, we can search for

$$\operatorname{argmin}_{\theta} \frac{1}{n} \sum_{i=1}^n \left(y^{(i)} - h_{\theta}(x^{(i)})\right)^2.$$

This is an example of **parametric learning**. Below we discuss a particular choice of $\mathcal{H}$ consisting of "linear" hypotheses.

⁵Why quadratic loss in the first place? It feels a bit like the choice was justified in hindsight (which I don't take issue with but I can't really figure it out).

- Develop the method
- Realise that convergence issues happen in the algorithms you devise and then
- go back to alter the loss to have properties that make things more convenient e.g. convexity.

3.3 Linear Regression

So far there's been no restriction on how to represent the hypothesis h i.e. $\mathcal{H}$ is a very broad class of functions. We'd like to, in some meaningful way, limit the family of hypotheses over which we'll learn. Making assumptions about the functional form of h will restrict $\mathcal{H}$. This section will assume that h is a linear hypothesis — the simplest form.

A **linear hypothesis** takes the form of an affine function

$$h_{\boldsymbol{\theta}}(\mathbf{X}) = \theta_0 + \theta_1 X_1 + \cdots + \theta_p X_p,$$

where we collect the unknown quantities θ_j , called the **parameters** (or **weights**) of our model, into a vector

$$\boldsymbol{\theta} = \begin{bmatrix} \theta_0 \\ \vdots \\ \theta_p \end{bmatrix} \in \mathbb{R}^{p+1}.$$

The linearity of our hypothesis refers to **linearity in the coefficients** θ_j .

Example 3.3.1

- $h_{\boldsymbol{\theta}}(X) = \theta_0 + \theta_1 X^3$ is linear in $\boldsymbol{\theta} = [\theta_0 \ \theta_1]^\top \in \mathbb{R}^2$,
- but $h_{\boldsymbol{\theta}}(X) = \theta_0 + \sin(\theta_1)X$ is not linear in $\boldsymbol{\theta}$.



Such a function $h_{\boldsymbol{\theta}}$ is also called a **linear predictor**, or a **discriminant function**. For notational convenience, one can define an intercept term $X_0 = 1$ so that h may be re-written as

$$h_{\boldsymbol{\theta}}(\mathbf{X}) = \langle \boldsymbol{\theta}, \mathbf{X} \rangle := \sum_{i=0}^p \theta_i X_i = \mathbf{X}^\top \boldsymbol{\theta},$$

where $\mathbf{X} = (X_0, X_1, \dots, X_p)$ is defined by $\mathbf{X}(\omega) = \begin{bmatrix} 1 \\ X_1(\omega) \\ \vdots \\ X_p(\omega) \end{bmatrix} \in \mathbb{R}^{p+1}$, and we call θ_0 a **bias** term.

A nice way to interpret a linear hypothesis is by the expression $\partial_j h(\mathbf{X}) = \theta_j$ i.e. holding all other predictors fixed, θ_j represents the effect on h of a one unit increase in X_j . This behaviour characterises a linear-response model.

A **linear response model** is suitable for situations where the response variable can vary, to a good approximation, indefinitely in either direction. More generally, a linear-response model works for any quantity that only varies by a relatively⁶ small amount compared to the variation in the predictive values e.g. human height.

These assumptions aren't suitable for some types of response variables and this will be discussed later under GLMs as a flexible generalisation of linear regression.

Here I collect the above assumptions to define a linear regression model for the conditional expectation of Y given $\mathbf{X}$.

⁶The same as linear approx in analysis: $\Delta x \approx 0 \implies \Delta y = f(x + \Delta x) - f(x) \approx 0$.

Let $(\mathbf{X}, Y) \sim \mathbb{P}_{\mathcal{X} \times \mathcal{Y}}$. A (population-level) **linear regression model** is a regression model

$$Y = f(\mathbf{X}) + \varepsilon = \mathbb{E}[Y | \mathbf{X}] + \varepsilon \quad \text{s.t.} \quad \mathbb{E}[\varepsilon | \mathbf{X}] = 0$$

that assumes linearity of the conditional mean i.e. that the conditional expectation of Y given $\mathbf{X}$ is a linear hypothesis h i.e. that there exists a vector of parameters $\boldsymbol{\theta} = [\theta_0 \ \cdots \ \theta_p]^\top$ s.t.

$$\mathbb{E}[Y | \mathbf{X}] = h_{\boldsymbol{\theta}}(\mathbf{X}) = \mathbf{X}^\top \boldsymbol{\theta}.$$

On the sample-level, we introduce more assumptions that allow us to relate how our data

$$\mathcal{T} = \{(x^{(i)}, y^{(i)})\}_1^n$$

is related to the population. As usual, we assume that our training set is a realisation of a random sample

$$(\mathbf{X}^{(1)}, Y^{(1)}), \dots, (\mathbf{X}^{(n)}, Y^{(n)}) \stackrel{\text{i.i.d.}}{\sim} \mathbb{P}_{\mathcal{X} \times \mathcal{Y}}.$$

We can think of these as identical copies of $(\mathbf{X}, Y)$ so the population assumptions hold for each $(\mathbf{X}^{(i)}, Y^{(i)})$. The linearity assumption can be written in matrix-form to summarise the n equations into one.

For the sake of simplifying notation, we'll collect the predictor random vectors into a single random vector (of random vectors)

$$\mathbf{X} = (\mathbf{X}^{(1)}, \dots, \mathbf{X}^{(n)}): (\Omega, \mathcal{F}) \rightarrow ((\mathbb{R}^{p+1})^n, \mathcal{B}_{(\mathbb{R}^{p+1})^n}).$$

A realisation of $\mathbf{X}$ is given by

$$\mathbf{X}(\omega) = \begin{bmatrix} \mathbf{X}^{(1)}(\omega) \\ \vdots \\ \mathbf{X}^{(n)}(\omega) \end{bmatrix} = \begin{bmatrix} x^{(1)} \\ \vdots \\ x^{(n)} \end{bmatrix}$$

Statistics textbooks present $\mathbf{X}(\omega)$ in a more presentable form by identifying this vector of column vectors $(\mathbb{R}^{p+1})^n$ with a matrix in $\mathbb{R}^{n \times (p+1)}$. Doing so allows one to succinctly represent linearity over all the training examples (a system of n equations) as a matrix equality. I'll denote the map that makes such an identification by Φ so a realisation of $\mathbf{X}(\omega)$ is identified with the matrix

$$\mathbb{X} := \Phi(\mathbf{X}(\omega)) = \begin{bmatrix} (x^{(1)})^\top \\ \vdots \\ (x^{(n)})^\top \end{bmatrix} = \begin{bmatrix} 1 & x_1^{(1)} & \cdots & x_p^{(1)} \\ \vdots & \vdots & \vdots & \vdots \\ 1 & x_1^{(n)} & \cdots & x_p^{(n)} \end{bmatrix}.$$

This matrix $\mathbb{X}$ whose rows are the input features of each example in $\mathcal{T}$ is called the **design matrix** of our model. When I need to speak about conditioning on a random variable I will refer to $\mathbf{X}$, and when I wish to speak of systems of equations (like I will immediately below) I will refer to $\mathbb{X}$. On the level of random variables, we have that

$$\Phi(\mathbf{X}) = \Phi(\mathbf{X}^{(1)}, \dots, \mathbf{X}^{(n)}) = \begin{bmatrix} (\mathbf{X}^{(1)})^\top \\ \vdots \\ (\mathbf{X}^{(n)})^\top \end{bmatrix}.$$

The rest of the notation is straight-forward. Simply collect the responses and errors into their own respective random vectors $\mathbf{Y}$ and $\boldsymbol{\varepsilon}$, and denote their realisations by labels $\mathbf{y}$ and residuals $\boldsymbol{\epsilon}$:

$$\mathbf{Y} = \begin{bmatrix} Y^{(1)} \\ \vdots \\ Y^{(n)} \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y^{(1)} \\ \vdots \\ y^{(n)} \end{bmatrix}, \quad \boldsymbol{\varepsilon} = \begin{bmatrix} \varepsilon^{(1)} \\ \vdots \\ \varepsilon^{(n)} \end{bmatrix}, \quad \text{and} \quad \boldsymbol{\epsilon} = \begin{bmatrix} \epsilon^{(1)} \\ \vdots \\ \epsilon^{(n)} \end{bmatrix}.$$

It follows that

$$\Phi(\mathbf{X})\boldsymbol{\theta} = \begin{bmatrix} (\mathbf{X}^{(1)})^\top \boldsymbol{\theta} \\ \vdots \\ (\mathbf{X}^{(n)})^\top \boldsymbol{\theta} \end{bmatrix}.$$

Finally, we may write all n equations as⁷

$$\mathbf{Y} = \Phi(\mathbf{X})\boldsymbol{\theta} + \boldsymbol{\varepsilon},$$

and so a realisation $\mathcal{T}$ of the experiment that these random variables formalise can be represented by the system:

$$\mathbf{y} = \mathbb{X}\boldsymbol{\theta} + \boldsymbol{\epsilon}.$$

3.3.1 EXTRA ASSUMPTIONS

2. Strict exogeneity: The errors have 0 mean conditional on the features i.e. for $i = 1, \dots, n$:

$$0 = \mathbb{E}[\varepsilon^{(i)} \mid \sigma(\mathbf{X}^{(1)}, \dots, \mathbf{X}^{(n)})].$$

3. No multicollinearity: The rank of the $n \times p$ regression matrix, X , is p with probability 1.
4. Constant conditional second moment of errors given predictors:
An assumption that the variances of all the error terms are equal (or similar):

$$\mathbb{E}[(\varepsilon^{(i)})^2 \mid \sigma(\mathbf{X}^{(1)}, \dots, \mathbf{X}^{(n)})] = \sigma^2 > 0.$$

5. No correlation between observations:

$$\mathbb{E}[\varepsilon^{(i)} \varepsilon^{(j)} \mid \sigma(\mathbf{X}^{(1)}, \dots, \mathbf{X}^{(n)})] = 0 \quad \text{for } i, j = 1, \dots, n \text{ where } i \neq j$$

Assumption 3 is left stated in the form Hayashi does (with respect to fixed design i.e. in the situation where one has the design matrix X after observing the random sample). I'm not yet sure what the random regressors version of the statement is.

Remarks 3.3.2

- Since we are assuming that our sample is i.i.d., in the strict exogeneity assumption we may condition on only one of the $\mathbf{X}^{(i)}$ instead of on the whole of $\sigma(\mathbf{X}^{(1)}, \dots, \mathbf{X}^{(n)})$ without gaining or losing any information.

Proof. Let $\mathcal{G}_n := \sigma(\mathbf{X}^{(1)}, \dots, \mathbf{X}^{(n)})$. Then $\sigma(\mathbf{X}^{(i)}) \subseteq \mathcal{G}_n$, and by the tower property:

$$\mathbb{E}[\varepsilon^{(i)} \mid \mathcal{G}_n] = \mathbb{E}[\mathbb{E}[\varepsilon^{(i)} \mid \sigma(\mathbf{X}^{(i)})] \mid \mathcal{G}_n] = \mathbb{E}(0 \mid \mathcal{G}_n) = 0$$

For the reverse implication, assume that $\mathbb{E}[\varepsilon^{(i)} \mid \mathcal{G}_n]$ and run the above argument backwards to conclude that $\mathbb{E}[\varepsilon^{(i)} \mid \sigma(\mathbf{X}^{(i)})] = 0$. ■

⁷Not only is this succinct but by *vectorising* the data in a programming language, we also benefit from linear algebra being well-optimised in built-in language packages/modules like `numpy` in Python. Such packages, in general, yield faster results than basic `for`-loops.

Economists prefer to write “ $\mathcal{G}_n$ ” because it inspires notation like $\mathbb{E}(\varepsilon | \mathbb{X})$ (found in Hayashi [1]) which is read as “fix the realised design matrix; the errors have mean zero.”

- When we combine assumptions 2 and 4, we get conditional homoskedasticity of our error terms:

Definition 3.3.3 The assumption of constant conditional variance

$$\text{Var}[\varepsilon^{(i)} | \sigma(\mathbf{X}^{(1)}, \dots, \mathbf{X}^{(n)})] = \sigma^2 > 0$$

of our errors given the predictors $\mathbf{X}$ is called **conditional homoskedasticity**.

Proof. For short, let $\sigma(\mathbf{X}^{(1)}, \dots, \mathbf{X}^{(n)}) =: \mathcal{G}_n$. Then,

$$\begin{aligned} \text{Var}[\varepsilon^{(i)} | \mathcal{G}_n] &= \mathbb{E}\left[\left(\varepsilon^{(i)} - \mathbb{E}[\varepsilon^{(i)} | \mathcal{G}_n]\right)^2 | \mathcal{G}_n\right] \\ &= \mathbb{E}\left[(\varepsilon^{(i)})^2 - \varepsilon^{(i)}\mathbb{E}[\varepsilon^{(i)} | \mathcal{G}_n] + \mathbb{E}[\varepsilon^{(i)} | \mathcal{G}_n]^2 | \mathcal{G}_n\right] \\ &= \mathbb{E}\left[(\varepsilon^{(i)})^2 | \mathcal{G}_n\right] - \underbrace{\mathbb{E}\left[\varepsilon^{(i)}\mathbb{E}[\varepsilon^{(i)} | \mathcal{G}_n] | \mathcal{G}_n\right]}_{\mathbb{E}[g(\mathbf{X}) | \mathcal{G}_n] = g(\mathbf{X})} + \mathbb{E}\left[\mathbb{E}[\varepsilon^{(i)} | \mathcal{G}_n]^2 | \mathcal{G}_n\right] \\ &= \mathbb{E}\left[(\varepsilon^{(i)})^2 | \mathcal{G}_n\right] - 2(\mathbb{E}[\varepsilon^{(i)} | \mathcal{G}_n])^2 + \mathbb{E}[\varepsilon^{(i)} | \mathcal{G}_n]^2 \\ &= \mathbb{E}\left[(\varepsilon^{(i)})^2 | \mathcal{G}_n\right] - \mathbb{E}[\varepsilon^{(i)} | \mathcal{G}_n]^2 \\ &= \sigma^2 - 0 \end{aligned}$$

■

- Strict exogeneity implies $\mathbb{E}(\varepsilon^{(i)}) = 0$ for all $i = 1, \dots, n$.

Proof. By the law of total expectation, $\mathbb{E}(\varepsilon^{(i)}) = \mathbb{E}(\mathbb{E}[\varepsilon^{(i)} | \sigma(\mathbf{X}^{(1)}, \dots, \mathbf{X}^{(n)})])$.

■

3.3.2 ORDINARY LEAST SQUARES (OLS)

Ordinary⁸ least squares is a type of linear least squares method for estimating the unknown parameters $\boldsymbol{\theta}$ in a classical linear regression model. The method minimises the sum of squared residuals of $\mathcal{T}$. We denote this sum by $J(\boldsymbol{\theta})$ and define it by:

$$J(\boldsymbol{\theta}) = \frac{1}{2} \sum_{i=1}^n \left(h_{\boldsymbol{\theta}}(x^{(i)}) - y^{(i)} \right)^2.$$

Note that the Hessian of J is positive-definite.

Had J been only convex in $\boldsymbol{\theta}$, we would only be able to conclude that any local minimiser $\hat{\boldsymbol{\theta}}$ of J is a global minimiser without any guarantee of $\hat{\boldsymbol{\theta}}$ (the argmin of J) being unique.

However, J is indeed strongly convex so this global minimiser $\hat{\boldsymbol{\theta}}$ is indeed unique, we call it the **OLS estimator** and by solving $\nabla_{\boldsymbol{\theta}} J = \mathbf{0}$ for the critical point, we obtain:

⁸There's some contention over what 'ordinary' actually means. The most sensible explanation I've found is from [this Stack Exchange comment](#) suggesting that it's there to contrast with the weight parameters in weighted least squares regression — a technique that can be found in the following subsection. Namely, if the weights in WLS are all equal to 1, you get OLS.

$$\hat{\boldsymbol{\theta}} = \underset{\boldsymbol{\theta}}{\operatorname{argmin}} J(\boldsymbol{\theta}) = (X^\top X)^{-1} X^\top \mathbf{y}.$$

The calculation is done by differentiating the cost function $J(\boldsymbol{\theta})$, setting it equal to zero to get what are known as the normal equations, and finding stationary points. Using matrix notation, we can rewrite⁹ $J(\boldsymbol{\theta})$:

$$J(\boldsymbol{\theta}) = \frac{1}{2}(\mathbf{X}\boldsymbol{\theta} - \mathbf{y})^\top (\mathbf{X}\boldsymbol{\theta} - \mathbf{y})$$

The relevant definitions for the following calculations can be found in the Matrix Calculus appendix at the end of these notes.

To minimise $J(\boldsymbol{\theta})$, one must calculate its derivative $\nabla_{\boldsymbol{\theta}}$ w.r.t. $\boldsymbol{\theta}$:

$$\begin{aligned} \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) &= \nabla_{\boldsymbol{\theta}} \left(\frac{1}{2}(\mathbf{X}\boldsymbol{\theta} - \mathbf{y})^\top (\mathbf{X}\boldsymbol{\theta} - \mathbf{y}) \right) \\ &= \frac{1}{2} \nabla_{\boldsymbol{\theta}} \left(\boldsymbol{\theta}^\top X^\top X \boldsymbol{\theta} - \boldsymbol{\theta}^\top X^\top \mathbf{y} - \mathbf{y}^\top X \boldsymbol{\theta} + \mathbf{y}^\top \mathbf{y} \right) \\ &= \frac{1}{2} \nabla_{\boldsymbol{\theta}} \operatorname{tr} \left(\boldsymbol{\theta}^\top X^\top X \boldsymbol{\theta} - \boldsymbol{\theta}^\top X^\top \mathbf{y} - \mathbf{y}^\top X \boldsymbol{\theta} + \mathbf{y}^\top \mathbf{y} \right) \\ &= \frac{1}{2} \nabla_{\boldsymbol{\theta}} \left[\operatorname{tr}(\boldsymbol{\theta}^\top X^\top X \boldsymbol{\theta}) - \operatorname{tr}(\mathbf{y}^\top X \boldsymbol{\theta}) - \operatorname{tr}(\mathbf{y}^\top X \boldsymbol{\theta}) + \operatorname{tr}(\mathbf{y}^\top \mathbf{y}) \right] \\ &= \frac{1}{2} \nabla_{\boldsymbol{\theta}} \operatorname{tr}(\boldsymbol{\theta}^\top X^\top X \boldsymbol{\theta}) - \nabla_{\boldsymbol{\theta}} \operatorname{tr}(\mathbf{y}^\top X \boldsymbol{\theta}) \\ &= \frac{1}{2} \nabla_{\boldsymbol{\theta}} \operatorname{tr}(\boldsymbol{\theta}^\top X^\top X \boldsymbol{\theta}) - X^\top \mathbf{y} \\ &\stackrel{(5)}{=} \frac{1}{2} ((X^\top X)^\top \boldsymbol{\theta} + X^\top X \boldsymbol{\theta}) - X^\top \mathbf{y} = X^\top X \boldsymbol{\theta} - X^\top \mathbf{y} \end{aligned}$$

In the third equality, we've introduced the trace because $\operatorname{tr}(a) = a$ for $a \in \mathbb{R}$. Furthermore, we have that $A^\top = \boldsymbol{\theta}$, $B = X^\top X$ and $C = I$ in (5). Set this to zero to find that $\hat{\boldsymbol{\theta}} = (X^\top X)^{-1} X^\top \mathbf{y}$. Thus, in one step you can find the value of $\boldsymbol{\theta}$ that minimises J .

Since X^{-1} appears in the calculation, the regressors must all be linearly independent i.e. the matrix X must have full column rank almost surely $\mathbb{P}(\operatorname{rank}(X) = p) = 1$. In the event that X is not invertible, this means that the input features are linearly dependent — it could be that a feature vector has been repeated. Otherwise, a pseudo-inverse can be used instead of X^{-1} .

There are other methods to approximate $\hat{\boldsymbol{\theta}}$. The following two algorithms are *iterative*:

3.3.3 GRADIENT DESCENT

Gradient descent is an iterative searching algorithm that searches for an optimal solution (minimiser) $\hat{\boldsymbol{\theta}}$ that minimises J . It begins by initialising some $\boldsymbol{\theta}$ and repeatedly performs the *simultaneous* update

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\boldsymbol{\theta}) \quad \text{for } j = 0, \dots, p.$$

(The symbol $:=$ here is for the assignment operator in code.)

Each iteration of this algorithm moves every θ_j in the direction of steepest decrease of J . This is because the partial derivative ∂_{θ_j} is defined as the direction of greatest increase of a function in the direction of θ_j .

The constant α is called the **learning rate**¹⁰ and determines how “large” the step is in each iteration.

⁹The $1/2$ is there to simplify future calculations.

¹⁰If all the features are normalised to $[-1, 1]$, then $\alpha = 0.01$ is a sensible value to start with.

More explicitly,

$$\begin{aligned}\frac{\partial}{\partial \theta_j} J(\boldsymbol{\theta}) &= \frac{1}{2} \frac{\partial}{\partial \theta_j} \sum_{i=1}^n \left(h_{\boldsymbol{\theta}}(x^{(i)}) - y^{(i)} \right)^2 \\ &= \sum_{i=1}^n \left(h_{\boldsymbol{\theta}}(x^{(i)}) - y^{(i)} \right) \frac{\partial}{\partial \theta_j} \left(h_{\boldsymbol{\theta}}(x^{(i)}) - y^{(i)} \right) \\ &= \sum_{i=1}^n \left(h_{\boldsymbol{\theta}}(x^{(i)}) - y^{(i)} \right) x_j^{(i)}\end{aligned}$$

Thus, the update rule looks at every example at each iteration and is called the **batch gradient descent update rule**:

$$\theta_j := \theta_j - \alpha \sum_{i=1}^n \left(h_{\boldsymbol{\theta}}(x^{(i)}) - y^{(i)} \right) x_j^{(i)} \quad \text{for } j = 0, \dots, p \quad \text{simultaneously.}$$

The batch gradient descent algorithm is as follows:

Algorithm 1 Batch gradient descent

```

1: procedure BATCH GRADIENT DESCENT
2:    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta}_0$  ▷ Initialise a guess for  $\boldsymbol{\theta}$ 
3:   while convergence criterion do
4:      $\theta_j \leftarrow \theta_j - \alpha \sum_{i=1}^n (h_{\boldsymbol{\theta}}(x^{(i)}) - y^{(i)}) x_j^{(i)}$  ▷ for every  $j$ 
5:   end while
6:   return  $\boldsymbol{\theta}$ 
7: end procedure

```

The magnitude of the update of θ_j is proportional to the error term $h_{\boldsymbol{\theta}}(x^{(i)}) - y^{(i)}$. If the algorithm encounters an example for which the prediction nearly matches the actual value of $y^{(i)}$, then $(h_{\boldsymbol{\theta}}(x^{(i)}) - y^{(i)}) \approx 0$ and it finds little need to change the parameters.

In general, gradient descent can be susceptible to local minima. This is because the algorithm makes locally optimal choices at each step/iteration. However, the way this particular OLS optimisation problem was set up guarantees convergence to a unique global minimiser (assuming α isn't too large). J has a unique global minimum.

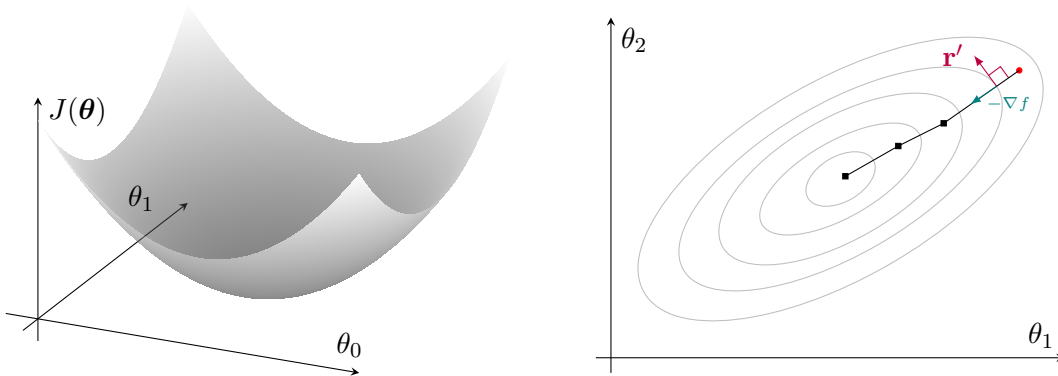


Figure 3.1: A paraboloid-shaped surface (left) with height $J(\boldsymbol{\theta})$ parameterised by $\boldsymbol{\theta} = [\theta_0, \theta_1]^\top$ and its level sets/contours (right). The contours for constant values of $J(\boldsymbol{\theta})$ are ellipses.

The direction of steepest ascent (of ∇f) is always orthogonal to the level set $f(\theta_1, \theta_2) = k$ of a surface i.e. for a parameterisation $\mathbf{r}$ of the level set, $\langle \nabla f, \mathbf{r}' \rangle = 0$. In the diagram, we follow $-\nabla f$ when updating our parameters. If α is too small, the algorithm will require far too many iterations. If the value of $J(\boldsymbol{\theta})$ increases then there's a very strong chance that α is too large and the minimiser has been overshoot. A disadvantage of batch gradient descent is that a single iteration can become very slow if the number n of training examples is very large.

3.3.4 STOCHASTIC GRADIENT DESCENT

Instead of scanning through all examples in a single iteration, a less computationally expensive option is to repeatedly loop over the training set $i = 1, \dots, n$, at each step using only the current example $(x^{(i)}, y^{(i)})$ to update the parameters.

The update rule for a single example $(x^{(i)}, y^{(i)})$ is called the Widrof-Hoff learning rule:

$$\theta_j := \theta_j - \alpha \left(h_{\boldsymbol{\theta}}(x^{(i)}) - y^{(i)} \right) x_j^{(i)}.$$

The stochastic gradient descent algorithm is as follows:

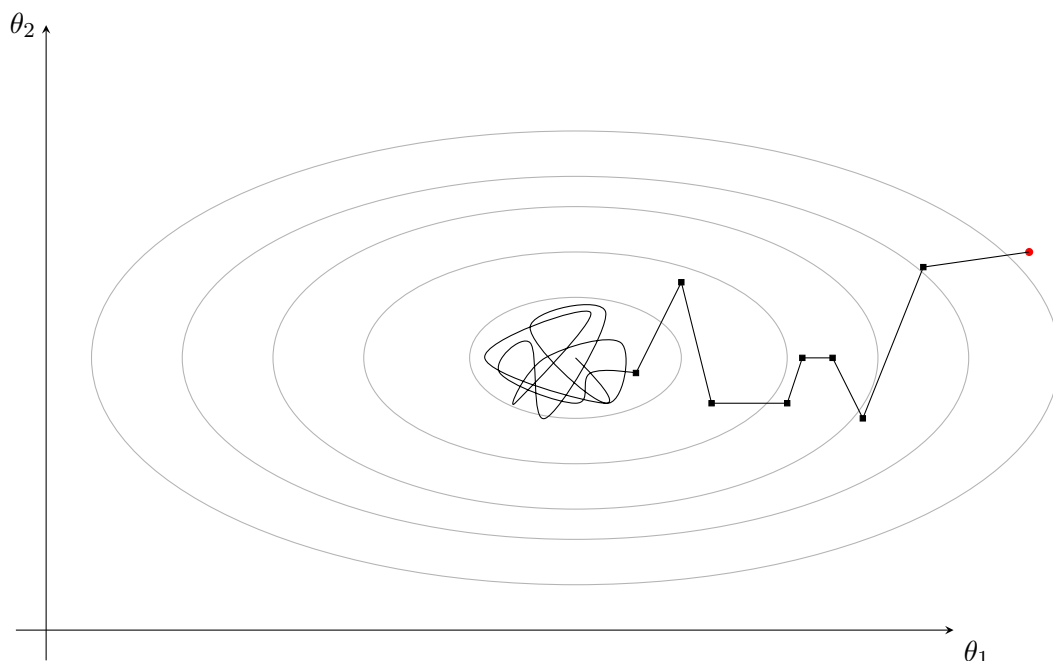
Algorithm 2 Stochastic Gradient Descent

```

1: procedure STOCHASTIC GRADIENT DESCENT
2:    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta}_0$  ▷ Initialise a guess for  $\boldsymbol{\theta}$ 
3:   for  $i \leftarrow 1, n$  do
4:      $\theta_j \leftarrow \theta_j - \alpha \left( h_{\boldsymbol{\theta}}(x^{(i)}) - y^{(i)} \right) x_j^{(i)}$  ▷ for every  $j$ 
5:   end for
6:   return  $\boldsymbol{\theta}_0$ 
7: end procedure

```

Since the algorithm uses the derivative for one example in each iteration, the parameters $\boldsymbol{\theta}$ are updated and improved slightly but may not be in the most direct direction downhill for J .

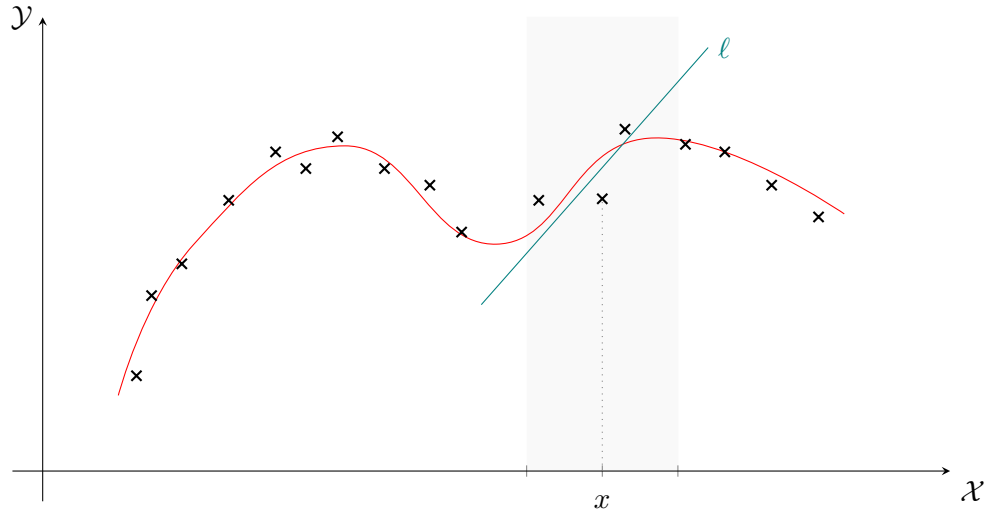


To this end, stochastic gradient descent takes a noisy route but on average gets $\boldsymbol{\theta}$ closer to

a minimum. Convergence is not guaranteed for this algorithm and θ will often keep oscillating around a minimum of $J(\theta)$. Decreasing the learning rate α in the later stages of the algorithm could yield good results.

3.4 Locally Weighted Linear Regression

Suppose that we have a set of data plotted below.



It's pretty clear what the shape of the data is. How does one fit a curve to the data that looks like **the red curve**? It's not so straightforward to find features (e.g. $\sqrt{x}$, $\log(x)$, $x^{2/3}$) to fit the data. Luckily, it's possible to sidestep this difficulty using a method called **locally weighted linear regression**:

The goal of ordinary least squares linear regression was to find optimal parameters $\hat{\theta}$ that minimise a cost function $J(\theta)$ (which quantifies how our hypothesis h_{θ} predictions differ from the given data) and then output a prediction model $h_{\hat{\theta}} = \hat{\theta}^T x$.

Locally weighted linear regression looks at a small neighbourhood around a query point x , fits a straight line ℓ to the values in this neighbourhood and uses ℓ to make a prediction. To fit said straight line to the data around such a point x , one can use a *modified cost function* $J_{\text{loc}}(\theta)$ which places greater emphasis on points close by to x e.g.

$$J_{\text{loc}}(\theta) := \sum_{i=1}^n \omega^{(i)} \left(\theta^T x^{(i)} - y^{(i)} \right)^2.$$

$\omega^{(i)}$ is known as a **weighting function** and for an example $(x^{(i)}, y^{(i)})$ exhibits the following:

- if $|x^{(i)} - x|$ is small, then $\omega^{(i)} \approx 1$
- else if $|x^{(i)} - x|$ is large, then $\omega^{(i)} \approx 0$

Thus, $\omega^{(i)}$ tells us how much attention must be paid to the values of $(x^{(i)}, y^{(i)})$ when fitting the straight line.

A common choice of $\omega^{(i)}$ is $\omega^{(i)} := \exp \left(-\frac{(x^{(i)} - x)^2}{2 \blacksquare^2} \right).$

WHAT IS THE QUANTITY $\blacksquare$ IN THE WEIGHTING FUNCTION?

We need some way to describe how quickly the importance (or weight) of a training example falls off from the query point x . We do so by incorporating a quantity $\blacksquare = \tau$ called the **bandwidth**. To illustrate, we graph $\omega^{(i)}$ against training examples:

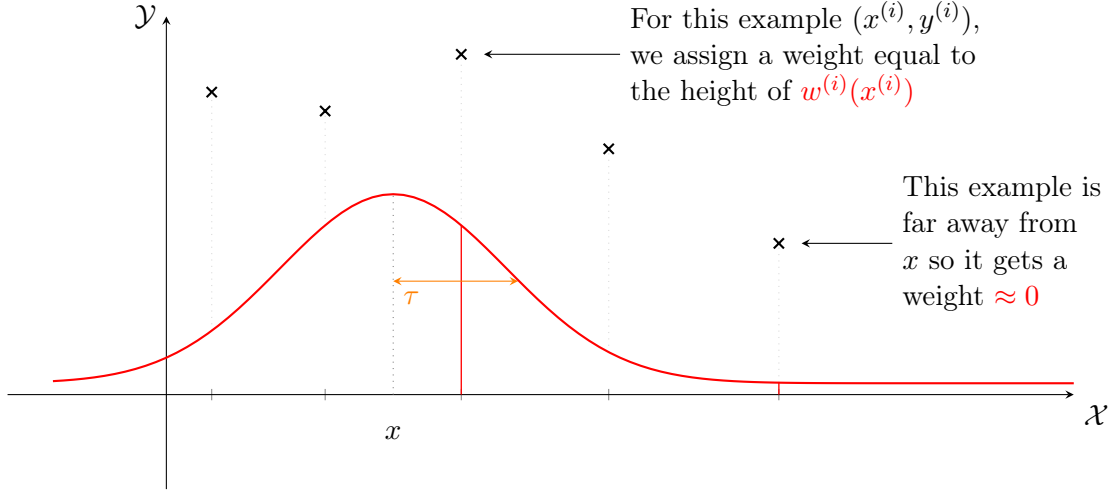


Figure 3.2: A visualisation of the weighting function (in red) for a neighbourhood of the query point x .

PARAMETRIC AND NON-PARAMETRIC ALGORITHMS

Unweighted linear regression is an example of a **parametric algorithm** because it has a fixed, finite number of parameters (the θ_i 's) which are fit to the data and once θ is fit and stored, the training data is no longer needed to make future predictions.

On the other hand, making predictions with locally weighted linear regression requires us to keep the entire training set around. It's an example of a *non-parametric* algorithm because the amount of data/parameters that must be kept in order to represent the hypothesis grows linearly with the size of the training data.

Non-parametric performs badly if one has a particularly large training set but a benefit is that the data can be fit quite well without having to manually fiddle with features.

3.5 Normality Assumptions (Ordinary Linear Regression)

A probabilistic assumption reveals that the ordinary least squares cost function $J(\theta)$ is the one we should be minimising because it naturally arises through the method of maximum likelihood estimation. We add this assumption to the linear regression model:

5. Normality of errors:

The distribution of $\varepsilon = (\varepsilon^{(1)}, \dots, \varepsilon^{(n)})$ conditional on $\mathbf{X}^{(1)}, \dots, \mathbf{X}^{(n)}$ is jointly normal $\mathcal{N}(\mathbf{0}, \sigma^2 I_n)$.

With the inclusion of joint normality, a linear regression model is termed an **ordinary linear regression model**.

We always assume that our random variables map into nice enough spaces (e.g. Borel spaces) s.t. there exists a system of regular conditional probabilities of $\varepsilon \in \text{Meas}_{\mathcal{F}, \mathcal{B}_{\mathbb{R}^n}}(\Omega; \mathbb{R}^n)$ given $\mathbf{X} := (\mathbf{X}^{(1)}, \dots, \mathbf{X}^{(n)}) \in \text{Meas}_{\mathcal{F}, \mathcal{B}_{(\mathbb{R}^{(p+1)})^n}}(\Omega; (\mathbb{R}^{(p+1)})^n)$ i.e. a map $\kappa_{\mathbf{X}}^{\varepsilon}: \mathcal{B}_{\mathbb{R}^n} \times (\mathbb{R}^{(p+1)})^n \rightarrow [0, 1]$ s.t.

1. For every $\mathbf{x} = (x^{(1)}, \dots, x^{(n)}) \in (\mathbb{R}^{(p+1)})^n$, $\kappa_{\mathbf{x}}^{\varepsilon}(\cdot, \mathbf{x})$ is a probability measure on the codomain $(\mathbb{R}^n, \mathcal{B}_{\mathbb{R}^n})$ of ε .
2. For each $B \in \mathcal{B}_{\mathbb{R}^n}$, the mapping $\mathbf{x} \mapsto \kappa_{\mathbf{x}}^{\varepsilon}(B, \mathbf{x})$ is:

- an $\mathcal{B}_{(\mathbb{R}^{(p+1)})^n}$ -measurable function satisfying for $\mathbb{P}$ -a.e. $\omega \in \Omega$:

$$\mathbb{P}[\varepsilon^{-1}(B) \mid \sigma(\mathbf{X})](\omega) = \kappa_{\mathbf{X}(\omega)}^{\varepsilon}(B, \mathbf{X}(\omega)),$$

- and is $\mathbb{P}_X$ -integrable, satisfying for all $B \in \mathcal{B}_{\mathbb{R}^n}$ and $D \in \mathcal{B}_{(\mathbb{R}^{(p+1)})^n}$:

$$\mathbb{P}(\varepsilon^{-1}(B) \cap X^{-1}(D)) = \int_D \kappa_X^\varepsilon(B, \mathbf{x}) d\mathbb{P}_X(\mathbf{x}).$$

The conditional normality means that κ_X^ε satisfies for every $B \in \mathcal{B}_{\mathbb{R}^n}$:

$$\kappa_X^\varepsilon(B, \mathbf{x}) = \gamma_{n, \sigma^2}(B) \quad \text{for } \mathbb{P}_X\text{-a.e. } \mathbf{x} \in (\mathbb{R}^{(p+1)})^n,$$

where φ_{n, σ^2} is a multivariate normal probability distribution. In other words, the conditional law of ε given X has a multivariate Gaussian density with mean $\mathbf{0}$ and covariance matrix $\sigma^2 I_n$. Also, since there's no dependence of φ_{n, σ^2} on $\mathbf{x}$, our disintegration formula becomes

$$\mathbb{P}(\varepsilon^{-1}(B) \cap X^{-1}(D)) = \int_D \gamma_{n, \sigma^2}(B) d\mathbb{P}_X(\mathbf{x}) = \gamma_{n, \sigma^2}(B) \mathbb{P}_X(D)$$

and this factorisation demonstrates that ε and X are independent.

3.5.1 CONSEQUENCES OF JOINT CONDITIONAL NORMALITY OF OUR ERRORS

The assumption of joint conditional normality of our errors implies several statements:

Corollary 3.5.1 Each $\varepsilon^{(i)}$ is conditionally normally distributed given $X^{(i)}$ with the same mean 0 and variance σ^2 .

Proof. The idea is to push the conditional law of ε given X forward via the natural coordinate projection map π_i . Indeed, for each $\mathbf{x} \in (\mathbb{R}^{(p+1)})^n$:

$$\kappa_X^{\varepsilon^{(i)}}(\cdot, \mathbf{x}) := (\pi_i)_\# \kappa_X^\varepsilon(\cdot, \mathbf{x}) = (\pi_i)_\# \gamma_{n, \sigma^2}(\cdot) = \gamma_{1, \sigma^2}(\cdot)$$

because the marginal distribution of a multivariate normal is univariate normal with variance the $(i, i)^{\text{th}}$ entry in the covariance matrix. ■

Corollary 3.5.2 The $\varepsilon^{(i)}$ are conditionally independent given X .

Proof. This follows from the previous corollary by noting that for every $\mathbf{x}$ the joint conditional law of ε given X factorises into the following product measure of every marginal conditional distribution of $\varepsilon^{(i)}$ given X i.e.

$$\kappa_X^\varepsilon(\cdot, \mathbf{x}) = \gamma_{n, \sigma^2} = \bigotimes_{i=1}^n \gamma_{1, \sigma^2}(\cdot) = \bigotimes_{i=1}^n \kappa_X^{\varepsilon^{(i)}}(\cdot, \mathbf{x}).$$
■

Corollary 3.5.3 There are no conditional cross-variance terms.

Proof. ■

Corollary 3.5.4 $\mathbf{Y} = \Phi(\mathbf{X})\boldsymbol{\theta} + \boldsymbol{\varepsilon}$ is conditionally normally distributed (with mean $\Phi(\mathbf{x})\boldsymbol{\theta}$ and covariance matrix $\sigma^2 I_n$) given $\mathbf{X}$.

Proof. According to Section 18.3.2 from KEB103 [2, p. 205], we can in a sense “push-forward” the regular conditional probability of $\boldsymbol{\varepsilon}$ given $\mathbf{X}$ by a jointly-measurable map in order to obtain the conditional law of $\mathbf{Y}$.

Definition 3.5.5 Let $(E_X, \mathcal{E}_X)$ and $(E_Y, \mathcal{E}_Y)$ be measurable spaces. A **Markov (or transition) kernel from E_X to E_Y** is a map $\kappa: \mathcal{E}_Y \times E_X \rightarrow \mathbb{R}$ s.t.

- for every $x \in E_X$, $\kappa(\cdot, x)$ is a probability measure on $(E_Y, \mathcal{E}_Y)$, and
- for every $B \in \mathcal{E}_Y$, $\kappa(B, \cdot)$ is $\mathcal{E}_X$ -measurable.

Theorem 3.5.6 Given a jointly measurable map $Z: E_Y \times E_X \rightarrow E_Z$, the function $\kappa^Z: \mathcal{E}_Z \times E_X \rightarrow [0, 1]$ defined for $B \in \mathcal{E}_Z$ and $x \in E_X$ by

$$\kappa^Z(B, x) := \kappa(\{y \in E_Y: Z(y, x) \in B\}, x)$$

is a Markov kernel from E_X to E_Z .

It suffices to check that the map which takes $\boldsymbol{\varepsilon}$ and $\mathbf{X}$ to $\mathbf{Y}$ is jointly-measurable. Indeed, let

$$Z: \underbrace{\mathbb{R}^n}_{\boldsymbol{\varepsilon}} \times \underbrace{(\mathbb{R}^{(p+1)})^n}_{\mathbf{X}} \rightarrow \underbrace{\mathbb{R}^n}_{\mathbf{Y}}$$

be the affine transformation

$$\mathbf{y} = Z(\boldsymbol{\varepsilon}, \mathbf{x}) = \Phi(\mathbf{x})\boldsymbol{\theta} + \boldsymbol{\varepsilon}.$$

This is clearly jointly measurable. Thus, the conditional law of $\mathbf{Y}$ given $\mathbf{X}$ is the regular conditional probability $\kappa_{\mathbf{X}}^{\mathbf{Y}}: \mathcal{B}_{\mathbb{R}^n} \times (\mathbb{R}^{(p+1)})^n \rightarrow [0, 1]$ defined for any $B \in \mathcal{B}_{\mathbb{R}^n}$ and $\mathbf{x} \in (\mathbb{R}^{(p+1)})^n$ by

$$\begin{aligned} \kappa_{\mathbf{X}}^{\mathbf{Y}}(B, \mathbf{x}) &:= \kappa_{\mathbf{X}}^{\boldsymbol{\varepsilon}}((Z \circ \iota_{\mathbf{x}})^{-1}(B), \mathbf{x}) \\ &= \kappa_{\mathbf{X}}^{\boldsymbol{\varepsilon}}(\{\boldsymbol{\varepsilon} \in \mathbb{R}^n: (Z \circ \iota_{\mathbf{x}})(\boldsymbol{\varepsilon}) \in B\}, \mathbf{x}) \\ &= \kappa_{\mathbf{X}}^{\boldsymbol{\varepsilon}}(\{\boldsymbol{\varepsilon} \in \mathbb{R}^n: Z(\boldsymbol{\varepsilon}, \mathbf{x}) \in B\}, \mathbf{x}) \\ &= \kappa_{\mathbf{X}}^{\boldsymbol{\varepsilon}}(\{\boldsymbol{\varepsilon} \in \mathbb{R}^n: \Phi(\mathbf{x})\boldsymbol{\theta} + \boldsymbol{\varepsilon} \in B\}, \mathbf{x}) \\ &= \gamma_{n, \sigma^2}(\{\boldsymbol{\varepsilon} \in \mathbb{R}^n: \Phi(\mathbf{x})\boldsymbol{\theta} + \boldsymbol{\varepsilon} \in B\}) \\ &= \int_{\{\boldsymbol{\varepsilon} \in \mathbb{R}^n: \Phi(\mathbf{x})\boldsymbol{\theta} + \boldsymbol{\varepsilon} \in B\}} \varphi_{n, \sigma^2}(\boldsymbol{\varepsilon}) d\lambda_{\mathbb{R}^n}(\boldsymbol{\varepsilon}) \\ &= \int_{h^{-1}(B)} \varphi_{n, \sigma^2}(\boldsymbol{\varepsilon}) d\lambda_{\mathbb{R}^n}(\boldsymbol{\varepsilon}) \quad \text{where } \mathbf{u} = h(\boldsymbol{\varepsilon}) := \Phi(\mathbf{x})\boldsymbol{\theta} + \boldsymbol{\varepsilon} \\ &= \int_B \varphi_{n, \sigma^2}(\mathbf{u} - \Phi(\mathbf{x})\boldsymbol{\theta}) |\det(J_{h^{-1}})(\mathbf{u})| d\lambda_{\mathbb{R}^n}(\mathbf{u}) \\ &= \int_B \varphi_{n, \sigma^2}(\mathbf{u} - \Phi(\mathbf{x})\boldsymbol{\theta}) d\lambda_{\mathbb{R}^n}(\mathbf{u}) \\ &= \mathcal{N}(\Phi(\mathbf{x})\boldsymbol{\theta}, \sigma^2 I_n)(B) \end{aligned}$$

The penultimate line’s Jacobian determinant of $h^{-1}(\mathbf{u}) = \mathbf{u} - \Phi(\mathbf{x})\boldsymbol{\theta}$ is simply 1 because it’s a translation. ■

The conditional independence of the $Y^{(i)}$ given $\mathbf{X}$ follows in an entirely analogous way to the $\varepsilon^{(i)}$ given $\mathbf{X}$. This observation is crucial for the following section.

3.5.2 MAXIMUM LIKELIHOOD ESTIMATION

We begin with the assumption that our data $\mathcal{T}$ has already been observed from a random sample

$$(\mathbf{X}^{(1)}, Y^{(1)}), \dots, (\mathbf{X}^{(n)}, Y^{(n)}) \stackrel{\text{i.i.d.}}{\sim} \mathbb{P}_{\boldsymbol{\theta}}$$

whose distribution is a particular parametric distribution $\mathbb{P}_{\boldsymbol{\theta}}$ from a family $\{\mathbb{P}_{\boldsymbol{\theta}}\}_{\boldsymbol{\theta} \in \Theta}$, where Θ is some indexing set. For the purposes of what follows, we assume that this family of probability measures is dominated by a σ -finite measure ν . In the discrete case, ν will be the counting measure; and in the absolutely continuous case, ν is the Lebesgue measure. This domination assumption allows us to speak of the¹¹ density of $\mathbb{P}_{\boldsymbol{\theta}}$, and therefore speak of the central object of this section.

The method of maximum likelihood estimation is a search for optimal parameters $\boldsymbol{\theta}' \in \Theta$ for which it was most likely that our data had been sampled from $\mathbb{P}_{\boldsymbol{\theta}'}$. We do this by working with a [surrogate](#) quantity called the joint likelihood L of our data. We view it as a function of $\boldsymbol{\theta}$ in order to carry out the aforementioned optimisation.

Definition 3.5.7 The **joint likelihood function for $\boldsymbol{\theta}$ given our data $\mathcal{T}$** is the density of the joint distribution of our sample (with respect to a dominating measure ν) evaluated at the observed value $\mathcal{T}$.

$$L = \frac{d\mathbb{P}_{(\mathbf{X}^{(1)}, Y^{(1)}), \dots, (\mathbf{X}^{(n)}, Y^{(n)})}}{d\nu} \Big|_{\mathcal{T}}.$$

In our case, our random sample is i.i.d. and made up of (absolutely) continuous random vectors (with respect to λ), so we may take $\nu = \otimes^n \lambda$ and obtain¹² the following expression:

$$L(\boldsymbol{\theta}) = \frac{d\mathbb{P}_{(\mathbf{X}^{(1)}, Y^{(1)}), \dots, (\mathbf{X}^{(n)}, Y^{(n)})}}{d\otimes^n \lambda} \Big|_{\mathcal{T}} \stackrel{\text{i.i.d.}}{=} \left(\prod_1^n \frac{d\mathbb{P}_{\boldsymbol{\theta}}}{d\lambda} \right) \Big|_{\mathcal{T}} = \prod_{i=1}^n f_{\boldsymbol{\theta}}((x^{(i)}, y^{(i)}))$$

Now we decompose this joint density into the product of a conditional density and a marginal. This is useful for two reasons:

1. We've only made assumptions about the conditional distribution of our errors from which it follows that each $Y^{(i)}$ is conditionally normally distributed given $\mathbf{X}^{(i)}$.
2. The marginal density is independent of $\boldsymbol{\theta}$ and so optimising $L(\boldsymbol{\theta})$ is an equivalent problem to optimising the **conditional** likelihood:

I think it's important to emphasise that the likelihood is not a product of probabilities. Instead, it's a Radon-Nikodym derivative of the (joint) density of the random variable of interest with respect to a (product) dominating measure. In our case with a random sample, the independence implies that our derivative becomes a product of densities. In the case that the original random variable of interest is discrete, the density is indeed the probability mass function (and hence the probability itself). This means that the likelihood is the probability but simply viewed as a function of the parameters. In the general case, there is no such correspondence.



¹¹Technically this is defined up to ν -null sets but it's not a massive abuse of language to say 'the' with the tacit assumption that we're referring to a representative of an equivalence class of functions that differ ν -a.e.!

¹²I'm leaving out some details that I expect are true but not quite sure e.g. if $\mu_1, \dots, \mu_n$ are measures, and $\nu_1, \dots, \nu_n$ is a collection of σ -finite measures s.t for every i we have $\mu_i \ll \nu_i$, then

$$\frac{d(\mu_1 \otimes \dots \otimes \mu_n)}{d(\nu_1 \otimes \dots \otimes \nu_n)} = \prod_{i=1}^n \frac{d\mu_i}{d\nu_i}.$$

CHAPTER A

Matrix Calculus

For a function $f: \mathbb{R}^{m \times n} \rightarrow \mathbb{R}$, we define the derivative of f with respect to $A \in \mathbb{R}^{m \times n}$ by:

$$\nabla_A f(A) = \begin{bmatrix} \frac{\partial f}{\partial A_{11}} & \cdots & \frac{\partial f}{\partial A_{1n}} \\ \vdots & \ddots & \vdots \\ \frac{\partial f}{\partial A_{m1}} & \cdots & \frac{\partial f}{\partial A_{mn}} \end{bmatrix}.$$

As a special case for a vector $\boldsymbol{\theta} = [\theta_1, \dots, \theta_n]^\top \in \mathbb{R}^n$ and $J: \mathbb{R}^n \rightarrow \mathbb{R}$, we define

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \begin{bmatrix} \frac{\partial J}{\partial \theta_1} \\ \vdots \\ \frac{\partial J}{\partial \theta_n} \end{bmatrix}.$$

Some properties about the trace of a square matrix $\text{tr}(A) = \sum_i A_{ii}$.

- For any two matrices A, B s.t. AB is square, $\text{tr}(AB) = \text{tr}(BA)$. As a corollary, there's a cyclic nature to the trace $\text{tr}(ABC) = \text{tr}(BCA) = \text{tr}(CAB)$.
- $\text{tr}(A) = \text{tr}(A^T)$
- $\text{tr}(A + B) = \text{tr}(A) + \text{tr}(B)$
- $\text{tr}(aA) = a\text{tr}(A)$ for a scalar a .

Some properties of the matrix derivative:

- (1) $\nabla_A \text{tr}(AB) = B^T$
- (2) $\nabla_{A^T} f(A) = (\nabla_A f(A))^T$
- (3) $\nabla_A \text{tr}(ABA^T C) = CAB + C^T AB^T$
- (4) $\nabla_A \det(A) = \det(A) \cdot (A^{-1})^T$

The following is used in the derivation of the normal equation:
Combine (2) and (3) to make (5):

$$\begin{aligned} \nabla_{A^T} \text{tr}(ABA^T C) &\stackrel{(2)}{=} (\nabla_A \text{tr}(ABA^T C))^T \\ &\stackrel{(3)}{=} (CAB + C^T AB^T)^T \\ &= B^T A^T C^T + BA^T C \end{aligned} \tag{5}$$

From regular calculus, we know that $(d/dx)(x^2) = 2x$.

The matrix derivative trace analogue is $\nabla_A \text{tr}(AA^T C) = CA + C^T A$ which is similar to

$$\frac{d(a^2 c)}{da} = 2ac$$

Bibliography

- [1] Fumio Hayashi. *Econometrics*. Princeton University Press, 2000.
- [2] Khallil Ebrahim Benyattou. Conditional exkebtations. 2026.